

Excel Lite

Launch-readiness evaluation and promotion plan

PUBLIC LAUNCH	NOT READY
LOCAL DEMO	READY WITH CAVEATS

A controlled local demonstration is supported by strong engine, UI, native, and offline evidence. Public distribution remains blocked by a million-row reachability defect, silent unsaved-work loss, and an untrusted macOS package, with native CI and release infrastructure still incomplete.

148 registered functions	1,585 conformance cases	1,341 Rust tests	96.29% core coverage
-----------------------------	----------------------------	---------------------	-------------------------

AUDIT DATE	10 July 2026
REVISION	4f112c9305930e9c947e5d0a52b558b3af656dc7
PLATFORM	macOS arm64 Tauri 2 Rust Svelte 5

PROJECT CONTEXT

MOTIVATION A public, instantly legible test of Kensho’s parallel-agent model: enter a formula and anyone can judge the result, while 148 independent functions create meaningful fan-out.	REQUIREMENTS Local macOS app; Rust engine; 148 functions; automatic recalc; native and CSV files; undo/redo; million-row grid; objective conformance; signed offline distribution.
GOAL Ship a useful single-sheet spreadsheet and show that many agents can build concurrently behind stable interfaces without sacrificing correctness, reviewability, or reproducibility.	OUTCOME Local demo achieved: 148 functions, 1,585 conformance cases, 1,341 Rust tests, and 96.29% coverage. Public release remains blocked by product-safety and distribution gaps.

Contents

This report separates demonstrated behavior from launch claims and defines promotion through objective gates.

Shareable summary	3
Verdict	3
What is already strong	3
Launch blockers	5
High-priority gaps	6
Acceptance matrix	8
Reproduced verification	9
Artifact record	9
Recommended parallel workstreams	10
Promotion gates	10
Existing supporting evidence	11
Audit constraints	11

Shareable summary

Excel Lite has a substantial, unusually well-tested calculation core and is ready for a controlled local demonstration. Current main passes 1,585 conformance cases, 1,341 Rust tests, 38 frontend tests, a 96.29% function/evaluator coverage gate, offline verification, a fresh Tauri package build, and DMG integrity verification. The packaged app launches and its native calculation, file-dialog, and offline paths have direct evidence.

It is **not ready for a public macOS launch or a full v1-complete claim**. The largest product defect is that the virtual sheet's lower half is unreachable: the 1,048,576-row spacer is clamped to 16,777,216 CSS pixels, and the bottom of the live grid starts at row 524,273. There is also no protection against discarding unsaved work. On the release side, the app is ad-hoc signed, fails strict code-sign and Gatekeeper checks, has no notarization, has no proper macOS application icon, and is not built or exercised as a native app in CI. The spec's LibreOffice oracle regeneration is only a scaffold, native e2e and scrolling benchmarks are absent, and the repository has no release/tag/remote workflow or public-facing release documentation.

The correct next move is a parallel launch-hardening pass: grid scalability and data-loss prevention as product blockers; native e2e and performance gates; signing/notarization and release assets; oracle/security validation; then a clean, tagged release-candidate rehearsal.

Verdict

Release claim	Verdict	Rationale
Controlled local demo on this Mac	Ready with caveats	Core, UI shell, package build, native recalc, file dialog, and offline behavior have evidence. Avoid the inaccessible lower sheet and explain the browser preview's limitations.
Internal arm64 artifact shared with technical testers	Conditional	Works locally, but recipients must bypass normal trust expectations because the artifact is not Developer ID signed or notarized. Use only with explicit instructions and no irreplaceable workbook data.
Public macOS download	Not ready	Grid reachability, unsaved-work loss, signing/notarization, native CI/e2e, release provenance, and release assets are incomplete.
SPEC.md v1 definition of done	Not met	Several Slice 4/5 exit conditions and the mandated CI pipeline are not present or do not pass.
Standalone web application	Not a product target	The browser command client is a visual/development preview. It intentionally does not calculate formulas or perform file operations.

What is already strong

Calculation engine and correctness gates

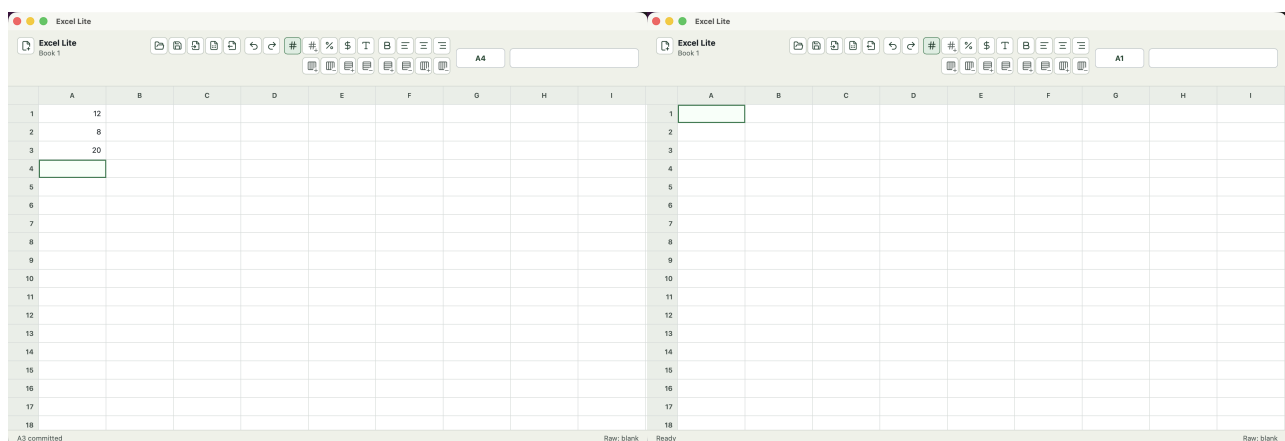
- **148 registered worksheet functions** are present with no missing or duplicate registry entries.
- **1,585 conformance cases across 152 TOML files** pass.
- Every dedicated function file meets the current automated minimum of five cases.
- **1,341 Rust tests** pass across core and app crates, including conformance, recalc hardening, cycle handling, reference shifting, native round-trip, formatting, resizing, and app adapter tests.
- The native round-trip property test is configured for **1,000 generated workbooks**.
- The measured function/evaluator line coverage is **96.29%**, above the 90% gate.
- Recalc tests cover direct propagation, cascades, diamonds, 1,000-cell fan-out, 10,000-cell incrementality, range dependency, and cycle cases under a one-second timeout.
- The headless xlite-core boundary remains free of Tauri and UI dependencies.

UI and native product evidence

- Svelte/TypeScript checking passes with **zero errors and zero warnings**.
- **38 frontend tests** pass across command clients, file operations, and grid logic.

- The production Vite bundle builds successfully.
- At 1120x760 and the configured 880x560 minimum, the live UI has no page overflow, no clipped controls, no header/grid/footer overlap, and no browser warning/error logs.
- The toolbar exposes accessible names and grouped roles; the visible grid exposes row, column, selected-cell, and formula-bar semantics.
- Browser interaction verified entry, commit, selection movement, toolbar undo/redo, and immediate repaint.
- The tracked final native verification shows A1=12, A2=8, and A3=20 for =A1+A2, plus the native Open dialog and packaged-app boot.
- A fresh build from the audited tree produced an app and arm64 DMG, and the freshly built app opened at the configured 1120x760 size.

Selected native evidence



Native recalc: A1=12, A2=8, A3=20

Packaged application boot at 1120x760

Offline and package evidence

- The bundle verifier passes its Tauri configuration, capability, resource, sample-workbook, frontend URL, and packaged-app URL checks.
- Runtime permissions are limited to Tauri core plus Open and Save dialogs; updater, HTTP, and shell plugins are absent.
- The bundled first-run `.xlite` sample is present inside the `.app`.
- The DMG verifies successfully with `hdiutil` and contains `Excel Lite.app` plus an `/Applications` symlink.
- Fresh artifact sizes are approximately **10 MB** for the app and **3.6 MB** for the DMG.
- The current binary is a thin **arm64** Mach-O executable.
- `npm audit` reports zero known vulnerabilities.
- A fresh RustSec scan reports zero vulnerability advisories. It does report 17 warnings, primarily unmaintained Linux GTK3 transitive dependencies and one unsound `glib` advisory; the `glib` path is not selected for the arm64 macOS target.

Launch blockers

P0 - The lower half of the sheet is unreachable

The UI declares 1,048,576 rows and constructs a virtual spacer of rows * 32px, or 33,554,432 pixels. In the live browser surface, the actual scroll height is clamped to **16,777,216 pixels**. At the absolute bottom:

- `scrollTop` is 16,776,708.
- The first rendered row is **524,273**.
- Only the normal 20 rows and 200 cells remain mounted, so virtualization itself is active.
- Rows roughly 524,293 through 1,048,576 cannot be reached.

This fails the million-row product claim and Slice 4 exit intent. A native WKWebView limit must also be measured; a browser-only fix is insufficient.

Required resolution

- Replace the single full-height spacer with segmented or normalized scrolling that never relies on a browser element taller than the platform limit.
- Add deterministic navigation tests for row 1, row 524,288, and row 1,048,576.
- Add tests for edits, selection, copy/paste, and viewport reads at the final row and final column.
- Add a native performance harness and enforce the 60 fps target or revise the spec to a measured, defensible threshold.

Release gate: the last row is reachable and editable in the packaged app, and the benchmark is repeatable in CI or a recorded release test.

P0 - Unsaved work can be discarded without warning

The UI has no dirty-workbook state, current-document path, close guard, or confirmation flow. New workbook clears an edited workbook immediately; the live browser check observed no confirmation dialog. Open and import operations also replace the current workbook without a save/discard/cancel decision.

For a spreadsheet, this is a direct data-loss path and should block a public release even though the original acceptance list did not spell it out.

Required resolution

- Track whether the workbook differs from the last successful new/open/save state.
- Protect New, Open, Import, window close, and app quit with Save / Discard / Cancel.
- Track the current `.xlite` path so ordinary Save does not always act as Save As.
- Define failure behavior for cancelled dialogs and failed saves without clearing dirty state.
- Add native e2e cases for every destructive transition.

Release gate: no destructive transition can silently discard a committed edit.

P0 - The macOS artifact is not distributable through the normal trust path

The fresh app reports:

- `Signature=adhoc`
- `TeamIdentifier=not set`
- `codesign --verify --deep --strict` fails with code has no resources but signature indicates they must be present
- `spctl --assess --type execute` fails with the same error
- no notarization ticket is stapled

This conflicts directly with goal G5 and Slice 5, both of which require a signed app/DMG.

Required resolution

- Build on macOS with a Developer ID Application identity and hardened runtime.
- Sign all nested code and resources, then verify strictly.
- Submit for Apple notarization and staple the result.
- Assess both the app and mounted DMG with Gatekeeper.
- Test a downloaded, quarantined artifact on a clean user account or clean Mac.

Release gate: strict code-sign verification, notarization validation, stapler validation, and Gatekeeper assessment all pass on the exact published artifact.

High-priority gaps

P1 - CI does not implement the release pipeline promised by the spec

SPEC.md requires the workflow to continue through a Tauri release build and native e2e smoke. The actual workflow ends after the frontend build and offline verifier on Ubuntu. It does not:

- `run npm run tauri:build;`
- launch the packaged app;
- execute the six native spreadsheet flows;
- verify DMG integrity;
- verify signing/notarization;
- publish checksums or release artifacts.

The local evidence is useful but cannot prevent regressions on future commits.

Release gate: a macOS CI/release workflow builds the same artifact that is published and blocks on native smoke, package integrity, signing, and notarization checks.

P1 - Slice 4 UI behavior is only partially implemented and verified

The current grid is a virtualized DOM table over a large spacer, not the specified canvas renderer. A DOM renderer is not inherently wrong, but the million-row failure demonstrates that the substitution has not met the original constraint. Additional spec behaviors are absent or incomplete:

- no Ctrl/Cmd+Arrow jump to the current region edge;
- no drag-to-select range interaction;
- Delete/Backspace clears only the active cell, not the selected range;
- the specified `clear_range` IPC command is absent;
- no date-format toolbar action, despite `dateIso` support in the command type;
- no automated native UI smoke suite;
- no recorded virtualization/frame-rate benchmark;
- no automated screen-reader or accessibility audit.

There is also minor command-contract drift: the spec declares `resize_column` returning no value, while the implementation returns a recalculation delta. The implementation may be the better contract, but the spec and code should agree before claiming completion.

Release gate: either implement the Slice 4 contract or explicitly revise and approve the spec, with native tests covering the chosen behavior.

P1 - The reference-oracle claim is not reproducible

The spec says LibreOffice Calc 24.8 can regenerate expected conformance values and that an advisory CI job performs this validation. In the repository:

- `scripts/regen_oracle.py` is explicitly a dry-run scaffold;
- `--write` always exits because the LibreOffice adapter is not implemented;
- LibreOffice is not installed in the audit environment;
- CI does not run an oracle job;
- the current validators prove schema, count, registration, deviations, and engine agreement with committed expectations, but not that all expectations came from LibreOffice;
- the five-case audit counts cases but does not prove every file covers nominal, boundary, error, coercion, and blank categories.

The 1,585 passing cases remain strong implementation evidence, but the stronger public claim that they are independently oracle-validated is not yet supported end to end.

Release gate: a pinned LibreOffice version regenerates or compares the corpus in automation, deviations are machine-reconciled, and the required semantic categories/composite/error quotas are reported.

P1 - Security hardening is incomplete

The runtime is commendably offline and narrowly permissioned, but Tauri's CSP is currently `null`, while the architecture explicitly calls for a strict CSP. There is no dependency audit in CI, SBOM, release threat model, or documented process for responding to advisories.

The fresh scans found no known npm or Rust vulnerability advisory affecting the macOS build. The Rust scan's target-independent warnings should still be triaged and recorded, particularly before claiming cross-platform support.

Release gate: a tested restrictive CSP is enabled, npm and Rust advisory scans run in CI with an explicit warning policy, and the release carries an SBOM or dependency inventory.

P1 - Public release identity and assets are missing

The package currently has development identity and minimal public metadata:

- bundle identifier `dev.kensho.excellite`;
- version `0.1.0` with no tag or changelog;
- no Git remote and no tags in this checkout;
- no release workflow;
- no project `LICENSE`, `SECURITY.md`, or support policy;
- a five-line README with no installation, system requirements, screenshots, data-format notes, or limitations;
- only a 32x32 PNG source icon;
- no `.icns` in the app, no `CFBundleIconFile`, and no application icon resource;
- only an arm64 artifact, with no documented Apple Silicon-only support decision.

Release gate: approved product identity, complete icon set, public documentation and license posture, a tagged clean source revision, published checksums, and an explicit architecture/support matrix.

Acceptance matrix

Area	Evidence	Status
148-function registry	Registry audit passes; no missing/duplicate/orphan functions	Pass
Conformance execution	1,585 cases across 152 files pass	Pass
Five cases per function	Automated dedicated-file count passes	Pass, narrow
Required case categories and quotas	Not fully machine-reported; oracle source is not reproducible	Evidence gap
Function/eval coverage	96.29% aggregate against a 90% gate	Pass
Recalc/incrementality	Dedicated direct, cascade, diamond, fan-out, 10k isolation, and range tests pass	Pass
Cycle behavior	Dedicated cases pass under one-second guards	Pass
Native round-trip	1,000-case property test plus fixed round-trip tests pass	Pass
CSV/XLSX behavior	Core/app fixtures and tests pass	Pass for scoped v1 behavior
Engine/UI decoupling	CI contract validation passes	Pass
Basic native workflow	Tracked native calculation, dialog, package boot evidence	Pass, manual
Native automated e2e	No Tauri driver/WebDriver suite in CI	Fail
Million-row navigation	Lower half is unreachable in live browser grid	Fail
60 fps virtualization target	No benchmark found	Not demonstrated
Unsaved-work safety	No dirty state or destructive-action confirmation	Fail
Offline runtime	Bundle URL scan, no updater/HTTP/shell plugin, local resources	Pass
CSP	Tauri config sets csp to null	Fail against architecture
Package creation	Fresh app and arm64 DMG build successfully	Pass
DMG integrity/layout	Checksum valid; app and Applications link present	Pass
Signing/notarization	Ad-hoc only; strict verification and Gatekeeper fail	Fail
Public release provenance	No remote, tag, release workflow, or clean tagged candidate	Fail
Release documentation/assets	README and icon set incomplete; no license/security docs	Fail

Reproduced verification

The following checks were run against the audited revision. All passed unless explicitly marked otherwise.

Check	Result
python3 scripts/validate_conformance_cases.py	Pass: 1,585 cases, 152 files
python3 scripts/audit_conformance_coverage.py	Pass: 148 functions complete; all dedicated files meet five-case count
python3 scripts/validate_ci_contracts.py	Pass
python3 -m unittest scripts/test_check_rust_coverage.py	Pass: 6 tests
cargo fmt --check	Pass
Repository CI-equivalent cargo clippy	Pass
cargo test --workspace	Pass: 1,341 tests
cargo llvm-cov plus repository gate	Pass: 96.29%
npm run check	Pass: 0 errors, 0 warnings
npm test	Pass: 38 tests, 3 files
npm run build	Pass
npm run verify:offline	Pass before and after native packaging
npm run tauri:build	Pass: app and arm64 DMG
hdiutil verify	Pass
Fresh packaged-app launch	Pass: 1120x760, Ready
Live browser layout at 1120x760 and 880x560	Pass: no clipping, overlap, page overflow, or console warnings/errors
Live bottom-of-sheet probe	Fail: capped at row 524,273 start
npm audit --json	Pass: zero vulnerabilities
fresh cargo audit	Zero vulnerability advisories; 17 warnings to triage
codesign --verify --deep --strict	Fail
spctl --assess --type execute	Fail
LibreOffice regeneration	Unavailable; script is a non-writing scaffold and LibreOffice is absent

Artifact record

Artifact	Value
DMG	target/release/bundle/dmg/Excel Lite_0.1.0_aarch64.dmg
DMG SHA-256	143e31c45b3e83c72e9734109807f53d8d18f2d0020368652278edf229602915
App executable	target/release/bundle/macos/Excel Lite.app/Contents/MacOS/xlite-app
Executable SHA-256	7f77af573e6f3353d76474a3e81f0a61e159d9306888a2074794e89e10bf131d
Architecture	arm64
Version	0.1.0
Bundle identifier	dev.kensho.excellite
Signature	ad-hoc, no Team ID

These hashes identify the locally audited artifacts only. They are not a release endorsement and should not be published as final until the artifact is rebuilt from a clean tag by the release workflow.

Recommended parallel workstreams

These tracks can proceed concurrently. Promotion should be gate-based, not date-based.

Track A - Grid scale and performance

Own segmented scrolling, final-row/final-column behavior, selection correctness at boundaries, and a native frame-time benchmark. This track closes the million-row blocker and decides whether the DOM-table implementation remains or the canvas architecture is restored.

Track B - Workbook safety and document lifecycle

Own dirty-state semantics, current path, Save versus Save As, destructive-action confirmation, window-close/app-quit handling, and failure recovery. Include native end-to-end coverage for every transition.

Track C - Native acceptance automation

Own the macOS CI lane and scripted packaged-app flows: formula entry, VLOOKUP, precedent recalc, undo/redo, save/reopen, CSV, XLSX smoke, dialog cancellation, and boundary navigation. The workflow must retain logs and screenshots on failure.

Track D - Distribution and release identity

Own the production bundle identifier, icon set, version/tag policy, arm64 versus universal support decision, Developer ID signing, notarization, DMG presentation, checksums, and clean-machine Gatekeeper test.

Track E - Oracle and security evidence

Own the LibreOffice adapter, pinned oracle comparison, case-category reporting, deviation reconciliation, CSP, dependency audits, SBOM, and a short release threat model.

Track F - Public documentation and demonstration package

Own README, license decision, security/support policy, installation steps, system requirements, limitations, native-format expectations, screenshots, sample workbook, demo script, changelog, and release notes. The browser preview must be labeled as a preview rather than a web edition.

Promotion gates

Gate 1 - Product safety

- Final row and column are reachable and editable.
- Unsaved work cannot be silently discarded.
- Required Slice 4 interactions are implemented or the spec revision is approved.
- Native performance and accessibility evidence is recorded.

Gate 2 - Reproducible acceptance

- All current core and UI gates remain green.
- LibreOffice comparison is reproducible.
- Native packaged-app e2e runs automatically.
- Security scans and CSP checks pass under an explicit policy.

Gate 3 - Release candidate

- Candidate is built from a clean, tagged revision.

- Product identity, icon, architecture matrix, docs, and license posture are complete.
- Checksums and build provenance are generated by automation.

Gate 4 - Distribution trust

- Developer ID signing passes strict verification.
- Notarization and stapling pass.
- Gatekeeper accepts the quarantined app and DMG on a clean environment.
- The exact candidate receives a final native smoke and offline-network check.

Only after all four gates should Excel Lite be described as a public v1 release.

Existing supporting evidence

- [Final UI and native product verification](#)
- [Native recalculation screenshot](#)
- [Packaged app screenshot](#)
- [Packaging and offline verification](#)
- [Conformance completeness audit](#)
- [Build-ready specification](#)

Audit constraints

- The repository had no Git remote or tags, so hosted CI history and published-release state could not be examined.
- The working tree contained pre-existing Kensho runtime/configuration changes and untracked orchestration state. Product source at the audited commit was not modified for this evaluation, but final release provenance must come from a clean checkout.
- The tracked native evidence is manual/AX-driven rather than a committed automated e2e suite.
- Rust advisory results are current as of the audit date and can change as the advisory database evolves.
- No claim is made that untested Excel semantics exactly match Microsoft Excel beyond the documented LibreOffice-based contract and deviations.